⁉️

Vize Notları

Sorumluluk Reddi Beyanı

> ❗ Valla ordan burdan toplama ve en temel konuları eksik olan, sadece kritik uç noktaları içeren, hatırlatma kafasında bi not oldu.

Pointers

Pointer Olarak Parametre Gönderme

- `head` parametresini `Node* head` olarak gönderdiğimizde `head` parametresinin bir kopyasını göndermiş oluruz.
- Bu kopya `head` ile listede sınırlı erişimimiz vardır. Ancak hala linked list üzerinde bazı işlermler yapabiliriz.
  - Bu işlemler `head`'in kendisini değiştirmez, sadece `head`'den *sonraki* düğümlerin `next` bağlantılarını değiştirir.
  - Sona eklemek için `temp` ile listenin sonunu bulursun (`while(temp->next != NULL)`).
    `temp->next = newNode;` yaptığında, `main`'deki listenin son elemanının `next`'ini değiştirmiş olursun. Bu **kalıcıdır**.
  - Silinecek düğümün *önceki* düğümünü (`prevNode`) bulman gerekir.
  - `prevNode->next = prevNode->next->next;` yaptığında, `main`'deki listenin yapısını **kalıcı olarak** değiştirirsin.

Pointer'ın Pointer'ı Olarak Gönderme

- `Node* head` bir kopyadır. Fonksiyon içinde `head = newNode;` yaparsan, `main`'deki `head` değil, *kopyası* değişir. Fonksiyondan çıktığın an bu değişiklik kaybolur.
- **İstisna:** Bu yüzden "Başa Ekleme" veya "Baştan Silme" yapacak `Node*` parametreli fonksiyonlar, yeni `head`'i `return` etmek zorundadır (`head = push(head, 10);` gibi).
- Eğer `return` kullanmak istemiyorsan, o zaman mecburen `Node** headRef` (çift pointer) kullanmalısın.

Graph Yapısı

- Tree'ler graph'ların alt kategorisidir.
- Tree'lerin hiyerşik bir yapısı varken graph'larda yoktur.
  - Örneğin graphlarda döngüler olabilir ancak tree'lerde olamaz
  - Graph'larda bir node'dan 2'den edge çıkabilir ancak tree'lerde çıkamaz.
- Kısaca aslen tree'ler, graph'ların kurallarla kısıtlanmış halleridir.
- Anlatılacak bir şeyi yok ama tree'lere çok odaklanıp sınavda karşına çıktığında "graph ne olaki" diye kalma diye yazıyorum.

BST | How Do You Kill a Parent of 2 Children?

- Silinecek node'un 2 dalı da doluysa;
  - Sol alt ağacın en büyük değerli düğümünü bul.
  - Eğer denge bozulmuş ise sağ alt ağacın en küçük elemanını bul.
  - Bu düğümdeki bilgileri silmek istediğimiz node'a aktar.
  - Bulduğumuz node'u sil.

AVL Tree

- Ağacın dengeli olması hedeflenmektedir.

- Dallar arasındaki mesafe max 1 katman olabilir.

Searching and Sorting Algorithms

Searching Algorithms

1. Linear Search

- Diziyi baştan sona kontrol eder, sıralı olması gerekmez.
- Karmaşıklığı $O(n)$'dir.

```
#include <stdio.h>
#include <stdlib.h>

// (Düğüm yapısını bildiğimizi varsayıyoruz)
typedef struct Node {
    int data;
    struct Node* next;
} Node;

Node* linearSearchLL(Node* head, int value) {
    Node* temp = head; // 1. Başlangıç noktası
    while (temp != NULL) { // 2. Liste bitene kadar dolaş
        if (temp->data == value) {
            return temp; // 3. Bulundu! Düğümün adresini dönd
ür
        }
        temp = temp->next; // 4. Sonraki düğüme geç
    }
    return NULL; // 5. Bulunmadı
}
```

2. Binary Search

- Sadece sıralı dizilerde çalışır.
- Dizinin ortasına yakın bir yerden başlar ve aranan değerle bulunulan değerin büyüklük karşılaştırmasına göre sola veya sağa gider.
- Karmaşıklığı $O(log(n))$'dir.

Sorting Algorithms

1. Bubble Sort

- Yan yana duran iki elemanı karşılatırır ve küçükten büyüğe sıralar.
- Her turda en büyük eleman sona baloncuk gibi gider.
- Karmaşıklığı $O(n^2)$'dir.

2. Selection Sort

- Her turda dizinin kalan kısmı içerisindeki en küçük elemanı bulur ve o anki turun başındaki elemanla yer değiştirir.
- Karmaşıklığı $O(n^2)$'dir.

3. Insertion Sort

- Dizinin başında sıralı bir alt liste oluşturur.
- Sıradaki elemanı alır ve bu alt liste içinde kaydırarak doğru yerine yerleştirir.
- Bağlı listeler için verimlidir çünkü kaydırma yerine sadece pointer'lar değişir.
- Karmaşıklığı $O(n^2)$'dir.

4. Quick Sort

- Divide and Qonquer yöntemini kullanır.
- Bir **pivot** eleman seçilir. (Genelde ortadan veya sondan)
- Pivottan küçük bütün elemanları pivotun soluna, büyük bütün elemanları pivotun sağına taşır.

- Sol ve sağ taraf için kendini recursive olarak çağırır.
- Genelde en hızlısıdır ve karmaşıklığı $O(n \times log(n))$'dir ancak zaten sıralı bir dizide $O(n^2)$ olabilir.

5. Merge Sort

- Diziyi ortadan ikiye böler ve her iki yarı için kendini recursive olarak çağırır. (Tek elemanlı dizi kalana kadar.)
- İki sıralı alt diziyi sıralı bir şekilde birleştirir.
- Her zaman $O(n \times long(n))$ garantisi verir ancak $O(n)$ ekstra bellek alanına ihtiyaç duyar.

Order Fonksiyonları

Preorder Fonksiyonu

- Kök → Sol → Sağ sıraysıyla gider.
- Ağacı kopyalamak veya yeniden oluşturmak için kullanılır.
    - Sıralaması ağacın haritasıyla aynı olduğu için bu tip işlerde tercih edilir.

```
void preorder(Node* node) {
    // 1. Temel Durum (Durdurma Koşulu)
    if (node == NULL) {
        return;
    }

    // 2. Özyineli Adım
    printf("%d ", node->data);  // ÖNCE KÖK'ü işle
    preorder(node->left);       // Sonra SOL'a git
    preorder(node->right);      // En son SAĞ'a git
}
```

```
//PREORDER KOPYA FONKSİYONU
Node* preorder(Node* head){
    if(head==NULL)return head;

    Node* tempNode = newNode(head->value);

    tempNode->left=preorder(head->left);
    tempNode->right=preorder(head->right);
    return tempNode;
}
```

Inorder Fonksiyonu

- Sol → **Kök** → Sağ sırlamasıyla gider.
- Sıralı veri elde etmek için kullanılır.
- Yapılan işlemler sonunda ağacın bozulup bozulmadığını kontrol ederken işlevseldir.
    - Eğer veri sıralı çıkmazsa ağaç bozulmuş demektir.

```
void inorder(Node* node) {
    // 1. Temel Durum
    if (node == NULL) {
        return;
    }

    // 2. Özyineli Adım
    inorder(node->left);        // Önce SOL'a git
    printf("%d ", node->data);  // Sonra KÖK'ü işle (ziyaret
et)
    inorder(node->right);       // En son SAĞ'a git
}
```

```
//INORDER YAZDIRMA FONKSİYONU
void inorder(Node* head){
    if(head==NULL)return;

    inorder(head->left);
    printf("%d ",head->value);
    inorder(head->right);
```

Postorder Fonksiyonu

- Sol → Sağ → **Kök** sırlamasıyla gider.
- Ağacı tamamen silmek için kullanılır.

```
void postorder(Node* node) {
    // 1. Temel Durum
    if (node == NULL) {
        return;
    }

    // 2. Özyineli Adım
    postorder(node->left);      // Önce SOL'a git
    postorder(node->right);     // Sonra SAĞ'a git
    printf("%d ", node->data);  // EN SON KÖK'ü işle
}
```

```
//POSTORDER SİLME FONKSİYONU
void postorder(Node** head){
    if(*(head)==NULL)return;

    postorder(&((*head)->left));
    postorder(&((*head)->right));
    free(*head);
}
```

Infix, Prefix, Postfix (Suffix)

- Matematiksel bir işlemi yazmanın 3 yolu vardır. Bu isimler, operatörün (`+`, `-`, `*`, `/`) işlenenlere (sayılara) göre nerede durduğunu belirtir.

Infix (Arada): Bizim her gün kullandığımız normal yazım şeklidir. Operatör, işlenenlerin **arasındadır**.

- Örnek: `a + b` veya `(2 + 3) * 5`

Prefix (Önde): Operatör, işlenenlerden **önce** gelir.

- Örnek: `+ a b` veya `* + 2 3 5`

Postfix (Sonda / Suffix): Operatör, işlenenlerden **sonra** gelir. Sınavda en çok bu çıkar. "Suffix" (son-ek) demenin sebebi budur.

- Örnek: `a b +` veya `2 3 + 5 *`

🤔 Neden Postfix/Prefix Kullanılır? (Sınav için Kritik Bilgi)

Infix (normal) gösterimin bilgisayarlar için 2 büyük sorunu vardır:

1. **İşlem Önceliği:** `2 + 3 * 5` yazdığında, önce çarpmanın mı toplamanın mı yapılacağını bilmen gerekir.
2. **Parantezler:** İşlem sırasını değiştirmek için parantezlere ihtiyaç duyarsın: `(2 + 3) * 5`.

Postfix ve Prefix notasyonlarının en büyük avantajı, parantezlere veya işlem önceliği kurallarına ihtiyaç duymamalarıdır. İfadenin yazılış sırası, işlem sırasını zaten net olarak belirtir.

- `(2 + 3) * 5` = `2 3 + 5 *` (Önce 2 ile 3'ü topla, sonra sonucu 5 ile çarp)
- `2 + (3 * 5)` = `2 3 5 * +` (Önce 3 ile 5'i çarp, sonra sonucu 2 ile topla)

Bilgisayarların bu ifadeleri işlemesi çok daha kolaydır ve bu işlemi **Stack (Yığıt)** kullanarak yaparlar.

Kural (Suffix):

1. İfadeyi soldan sağa doğru tara.

2. Eğer bir **sayı (işlenen)** görürsen, onu Stack'e `push` et.
3. Eğer bir **operatör (+, -, *, /)** görürsen:
   - Stack'ten 2 kez `pop` yap (ilk `pop` ettiğin `b`, ikinci `a` olsun).
   - İşlemi yap: `a operator b`.
   - İşlemin sonucunu tekrar Stack'e `push` et.
4. İfade bittiğinde, Stack'te kalan tek değer işlemin sonucudur.

> ❗ Prefix'te ise aynı işlemleri tersten gerçekleştirirsin.

❓ Sınav Sorusu 2: Infix İfadeyi Postfix'e Çevirme

Sana `(a + b) * c` gibi bir Infix ifade verilir, Postfix hali istenir. Burada da bir **operatör yığını (operatorStack)** kullanılır.

(Bu mantık `4.haftapptx.pdf` sayfa 88'de gösterilmiştir)

Kural:

1. Bir `cikti` (output) string'i ve bir `operatorStack`'i (operatör yığını) oluştur.
2. İfadeyi soldan sağa tara.
3. **Sayı/Harf (işlenen)** görürsen: Doğrudan `cikti`'ya ekle.
4. `(` (Açma parantezi) görürsen: `operatorStack`'e `push` et.
5. `)` (Kapama parantezi) görürsen:
   - `operatorStack`'ten `(` görene kadar `pop` ettiğin her operatörü `cikti`'ya ekle.
   - `(`'i de `pop` et ama `cikti`'ya *ekleme* (atla).
6. **Operatör (+, -, *, /)** görürsen:
   - Stack'in tepesindeki operatörün *işlem önceliği* seninkinden *büyük veya eşitse*, onu `pop` edip `cikti`'ya ekle. (Bunu, önceliği düşük bir operatör görene veya stack boşalana kadar yap).
   - Sonra, kendi operatörünü `operatorStack`'e `push` et.

7. İfade bitince: `operatorStack`'te ne kaldıysa hepsini `pop` edip `cikti`'ya ekle.

About @Me

Mertcan Gündoğan LinkedIn Profile

Mertcan (@mertcangndn) • Instagram profile

61 Followers, 109 Following, 10 Posts - See Instagram photos and videos from Mertcan (@mertcangndn)

https://www.instagram.com/mertcangndn/

Mertcangndn - Overview

Mertcangndn has 14 repositories available. Follow their code on GitHub.

https://github.com/Mertcangndn